ASSIGNMENT – UNIT 4

NoSQL Database: Column-Family Databases, HBase & Cassandra

Answers to all 15 questions

Q1. Explain the concept of Column-Oriented NoSQL Databases. How does the column-family model differ from the traditional relational table model? Discuss advantages and limitations for large-scale applications.

Column-oriented NoSQL databases store data in rows identified by keys, with related columns grouped into column families. They are designed for distributed storage and high-volume workloads. In HBase, a row can contain different qualifiers and versions; Cassandra uses partition keys, clustering columns and tables designed around queries.

In an RDBMS, tables generally have a fixed schema, each row follows the same columns, and relationships are often represented using foreign keys and joins. In a column-family store, records can be sparse or evolve over time, and data is commonly partitioned across nodes.

Advantages: horizontal scaling, high write throughput, flexible attributes, fault tolerance through distribution/replication, and efficient access to large datasets by key. Limitations: query patterns must be planned carefully, joins and ad-hoc queries are limited compared with SQL systems, data duplication may be necessary, and operations/consistency settings can add complexity.

Q2. Compare Apache HBase and Apache Cassandra. Which would you choose for a highly distributed application and why?

HBase is built on HDFS and commonly uses HMaster, RegionServers and ZooKeeper. It provides strongly consistent reads and writes at the row level, with automatic region splitting and distribution. Cassandra uses a peer-to-peer ring architecture; any node can coordinate a request, and replication plus tunable consistency levels provide availability and consistency trade-offs.

Data model: HBase uses sorted row keys, column families, qualifiers, timestamps and versions. Cassandra uses tables with partition keys and clustering columns, queried with CQL. Query processing: HBase commonly uses Get, Put, Scan, Delete and filters through APIs or shell; Cassandra supports CQL but queries should follow the primary-key design. Both scale horizontally and tolerate node failures, but Cassandra has built-in multi-replica, multi-data-center design and tunable consistency.

For a highly distributed, geographically dispersed application requiring continuous availability and predictable key-based queries, I would generally choose Cassandra. For Hadoop/HDFS integration, very large sparse tables and strongly consistent row-level operations, HBase may be a better fit. The choice depends on access patterns and operational requirements.

Q3. Explain HBase architecture and the roles of HMaster, RegionServer, ZooKeeper, HDFS, Regions and MemStore. Include a diagram.

HMaster coordinates cluster administration: assigning regions, balancing load and managing schema operations. It is not normally in the data path for every read/write. RegionServers host regions and serve client requests. A region is a contiguous range of table rows ordered by row key; large regions can split. ZooKeeper helps coordinate cluster state and server discovery. HDFS stores durable HBase files (HFiles) and write-ahead logs (WALs). MemStore buffers recent writes in memory before flushing them to HFiles.

Write path: the client locates the region, sends a Put to its RegionServer, the mutation is recorded in the WAL, then applied to MemStore. Later, MemStore flushes sorted data to HFiles on HDFS. Compactions merge HFiles over time. Read path: a RegionServer checks MemStore and relevant block caches/HFiles, reconciles cells and versions, and returns results. HMaster handles region assignment and recovery after failures.

Architecture diagram:

```
Client --> ZooKeeper / metadata lookup --> RegionServer
                                        |--> MemStore + WAL
                                        |--> HDFS / HFiles
HMaster --> region assignment, balancing and recovery
```

Client → ZooKeeper/metadata lookup → RegionServer → MemStore + WAL → HDFS/HFiles
↑
HMaster
(assigns/balances regions)

Q4. Explain the Column-Family Data Store model using row key, column family, qualifier, cell, timestamp and versioning.

A row key uniquely identifies a row and determines its sorted position. A column family groups related columns and is defined when an HBase table is created. A column qualifier names an individual column inside a family and can vary by row. A cell is the value at the intersection of row key, family, qualifier and timestamp. Timestamps allow multiple versions of a cell to be retained according to version settings.

Example: row key S101; family personal has name=Ayushi and city=Roorkee; family academic has course=BCA and marks=88. A cell may be represented as (S101, academic:marks, timestamp, 88). In an RDBMS, columns are usually declared in a fixed table schema and each row follows that structure. HBase supports sparse, versioned columns and stores related column-family data together. Cassandra's column-family terminology is often used historically, but modern Cassandra tables are defined with a primary key and CQL columns.

Q5. Design an HBase model for billions of student activity records with variable attributes.

Use a table such as StudentActivity with column families: profile (stable student details), activity (activity type, course, score, status) and context (device, source, optional attributes). Keep the number of column families small because each family has separate storage and flush/compaction behavior.

A row key must distribute writes while supporting expected lookups. For example, use a salted or hashed prefix plus student ID and reverse/encoded event time: [hashPrefix]#[studentId]#[reverseTimestamp]#[eventId]. This spreads traffic across regions and groups a student's recent events together. If scans by time are common, maintain a separate time-bucketed table keyed by [dateBucket]#[salt]#[timestamp]#[eventId]. Avoid a monotonically increasing timestamp at the beginning of the key, which can create a hotspot. Use TTL if old activity expires and pre-split regions where workload estimates justify it.

Q6. Discuss consistency and transactions in HBase and Cassandra. Why do distributed databases make different trade-offs from relational databases?

HBase provides strongly consistent reads and writes for a single row, but it does not provide general multi-row ACID transactions by default. Applications needing cross-row atomicity require additional design or systems. Cassandra offers tunable consistency such as ONE, QUORUM and ALL. With replication factor N, read/write consistency levels determine how many replicas must respond; QUORUM often balances consistency and availability. Lightweight transactions using Paxos support conditional updates but cost more latency.

Traditional relational databases often emphasize ACID transactions across related rows. Distributed NoSQL systems prioritize partitioning, throughput, availability and predictable scaling, so they commonly narrow transaction scope or expose consistency choices. Applications should define what must be immediately correct, whether stale reads are acceptable, and how retries, idempotency and concurrent updates are handled.

Q7. Explain how Cassandra achieves high availability using replication and consistency levels.

Cassandra distributes partitions across nodes using a partitioner and replicates each partition according to the keyspace replication strategy. In a multi-data-center deployment, NetworkTopologyStrategy can place replicas across racks and data centers. There is no permanent master for all writes; a coordinator node routes requests to replicas. If a node is unavailable, other replicas may still serve requests.

Consistency level controls how many replicas must acknowledge an operation. ONE requires one replica; QUORUM requires a majority of replicas for the relevant replica set; ALL requires every replica and therefore is less available during failures. For common configurations, read and write quorum intersection helps ensure reads observe acknowledged writes, subject to correct configuration and repair practices. Hinted handoff, read repair and anti-entropy repair help replicas converge. Availability is not unlimited: the chosen consistency level, replica placement and number of failed nodes determine whether an operation succeeds.

Q8. Compare major query features of HBase and Cassandra. Why are data modeling and query design important?

HBase provides Get for a row key, Put for insert/update, Delete, Scan over row-key ranges, counters and filters. Applications often use Java APIs or the HBase shell; secondary-index and relational-style query capabilities are limited unless added separately. Cassandra provides CQL commands such as INSERT, SELECT, UPDATE and DELETE, plus filtering and aggregation features with constraints. Efficient queries usually specify the partition key and, where relevant, clustering columns.

Both databases organize data around keys and partitioning. Poor key design can cause full scans, uneven partitions, hotspots or excessive latency. Cassandra commonly duplicates data into separate tables for different query patterns. HBase row keys must support the desired point lookups and scans. Model the queries first, define key structure and partition sizes, then choose families/columns and indexes only where supported and justified.

Q9. Explain how HBase and Cassandra scale as data and requests increase.

Both scale horizontally by adding machines, but distribute work differently. HBase splits tables into regions by row-key range and assigns regions to RegionServers. HDFS stores data across DataNodes. Adding RegionServers can spread regions and increase serving capacity; region splitting and balancing help distribute load. HBase availability depends on HDFS, WAL recovery, ZooKeeper and correct region reassignment.

Cassandra partitions rows by partition key and replicates partitions across nodes. Adding nodes allows token ranges and replicas to be redistributed, increasing storage and request capacity. Any node can coordinate a request. Cassandra supports multi-data-center replication and configurable consistency. In either system, adding nodes cannot fix a bad key design: hot keys, oversized partitions, skewed ranges or inefficient scans can remain bottlenecks. Capacity planning, monitoring and balanced partitioning are essential.

Q10. Design a conceptual model for a real-time event logging system generating millions of events per minute.

A column-family database is suitable because events are append-heavy, high volume, time-oriented and often have optional fields. In Cassandra, use a table such as EventsByUserDay with partition key ((user_id, event_date), event_time, event_id) and columns event_type, status, source, payload. A second table EventsByTypeDay may support type-based queries. Keep partitions bounded by day/hour or another suitable time bucket and avoid unbounded partitions.

Use replication factor 3 across racks or data centers. Choose a write consistency level such as LOCAL_QUORUM for stronger local durability or LOCAL_ONE when low latency/availability is the priority and occasional stale reads are acceptable. Batch only related writes when appropriate; do not use large cross-partition batches as a throughput shortcut. Scale by adding nodes and monitor partition sizes, write latency, compaction and dropped messages. Apply retention/TTL if events expire.

Q11. Explain how column-family databases support CMS and blogging platforms, including evolving structures.

A CMS can store article data in a table keyed by article ID, with columns for title, body reference, author, category, publication status, created time and optional metadata. A separate table keyed by author ID can list authored articles; another keyed by tag can list article IDs. A blogging platform may use BlogById, BlogsByAuthor and BlogsByTag tables to support different access patterns. HBase can group stable fields into a small number of families and use qualifiers for optional metadata.

The main challenge is that column-family databases do not provide flexible relational joins across all these entities. Data is often duplicated to make reads fast, and updates must keep duplicated views synchronized. Evolving attributes are easier than in a rigid schema, but application code must validate data, manage schema evolution, handle missing fields and decide which query paths need their own table or index.

Q12. Explain Cassandra counters and problems with concurrent updates.

Cassandra supports counter columns for incrementing or decrementing distributed counts. A counter table uses a primary key to identify the counter, and updates use syntax such as UPDATE PageCounts SET views = views + 1 WHERE page_id = 'home';. Counter updates are special operations and should not be treated like ordinary numeric assignments. They have restrictions, including limited support for mixing counters with non-counter columns and limitations around batches and idempotency.

Concurrent increments are coordinated across replicas, but retries and timeouts make client-side assumptions risky. Counter operations are not idempotent: retrying an increment may count it twice if the original succeeded but the acknowledgement was lost. Counters can also have higher operational overhead and consistency constraints. For exact business totals, consider event records plus periodic aggregation or an application-level deduplication design. For approximate/high-throughput metrics, counters may be suitable after testing the workload.

Q13. Explain TTL in Cassandra/HBase for sessions that expire after 24 hours.

TTL assigns a lifetime to stored data. In Cassandra, INSERT or UPDATE can use USING TTL 86400 to expire a value after 24 hours; table-level default TTL is also possible. Expired cells become invisible and are eventually removed during compaction. In HBase, TTL can be configured on a column family in seconds; cells older than the TTL are excluded and removed during compaction.

For sessions, store session ID as the key and fields such as user ID, login time and status. Set TTL to 86,400 seconds and refresh it when session activity should extend the lifetime. TTL reduces the need for manual cleanup, but expiration is not necessarily an immediate physical deletion. Account for clock differences, compaction/storage reclamation, and the fact that TTL on fields may expire independently in Cassandra. Do not rely on TTL alone for security-sensitive session revocation; validate expiration at the application layer too.

Q14. Recommend HBase or Cassandra for a real-time analytics platform with high writes, large datasets, global users and node failures.

For a platform with globally distributed users, continuous writes, multi-data-center resilience and predictable key-based reads, Cassandra is often the stronger default. Its peer-to-peer architecture, replication strategies and tunable consistency allow local reads/writes and continued operation during some node failures. Adding nodes can expand capacity, provided partition keys distribute traffic evenly.

HBase is attractive when analytics integrates closely with Hadoop/HDFS, needs ordered row-key range scans, and benefits from strong row-level consistency. It is not inherently a full SQL analytics engine; broader analysis may be done through Spark or other ecosystem tools. Before choosing, confirm query patterns: Cassandra needs tables designed for each query; HBase needs row keys that support point access or scans. For arbitrary ad-hoc aggregation, neither should be selected without an analytics layer. Given the stated global availability and failure-tolerance needs, I recommend Cassandra with NetworkTopologyStrategy, carefully chosen LOCAL_QUORUM/LOCAL_ONE levels, replication across data centers, and bounded time-bucketed partitions.

Q15. Critically analyze why careful data modeling is essential in HBase and Cassandra.

Column-family databases provide flexible schemas and horizontal scaling, but performance depends heavily on key and query design. In HBase, a sequential row key such as a timestamp first can direct writes to one region and create a hotspot. Poorly chosen row keys can make needed scans expensive. Too many column families increase storage/flush/compaction overhead; too few or incorrectly grouped families can make access patterns inefficient. Region sizes and pre-splitting also affect load balance.

In Cassandra, a partition key that has low cardinality or receives most writes can create hot nodes. Very large partitions increase memory, compaction and read costs; tiny partitions create overhead. Querying without a partition key may require filtering or a full scan and perform poorly. Incorrect replication factors or consistency levels can weaken availability or increase latency. Denormalization duplicates data and requires coordinated updates. Therefore, begin with required queries and consistency needs, choose balanced keys and bounded partitions, select replication and TTL deliberately, and validate using realistic load tests and monitoring.